Supplementary Information

This document provides supplementary code excerpts referenced in the article titled **"Detecting and Mapping Manufactured Housing Communities."** These scripts illustrate the key computational steps used in the data processing, model training, and result generation described in the paper. The code is organized by workflow stage and includes comments for clarity and reproducibility. All scripts were written in Python 3.11 and executed on a standard desktop PC. For full scripts, dependencies, and updates, refer to the GitHub repository: https://github.com/arminyeganeh/mhc. Source Code 1 divides the State of Wisconsin into smaller tiles using a fixed latitude increment (0.001 degrees), calculates the corresponding longitude width for each tile, and saves the resulting coordinates to a CSV file.

Source Code 1 Script to generate bounding boxes for a specified region.

In [ ]:
```python
import csv # Used for writing CSV files.
import os # Used for creating directories and file path operations.
import math # Used for performing calculations.

# Define the bounding box coordinates for Wisconsin
wi_bbox = (47, 42.45, -93, -87.5)

# Calculate the number of tiles needed
tile_height = 0.001  # One-thousandth of a degree latitude

# Create a list to store the bounding box data
bounding_box_data = []

# Calculate the width of a tile based on latitude
for y in range(int((wi_bbox[0] - wi_bbox[1]) / tile_height)):
    latitude = wi_bbox[0] - y * tile_height
    tile_width = tile_height / abs(math.cos(math.radians(latitude)))
    num_tiles_longitude = int((wi_bbox[3] - wi_bbox[2]) / tile_width)

    for x in range(num_tiles_longitude):
        tile_bbox = (
            latitude,
            latitude - tile_height,
            wi_bbox[2] + x * tile_width,
            wi_bbox[2] + (x + 1) * tile_width,
        )
        # Append the data to the list
        bounding_box_data.append((x, y, *tile_bbox))

# Write data to CSV file
csv_file_name = "E:/wi/wi_bounding_boxes.csv"
with open(csv_file_name, mode="w", newline="") as file:
   writer = csv.writer(file)
   writer.writerow(["x", "y", "north", "south", "west", "east"])  # Writing header
   writer.writerows(bounding_box_data)  # Writing data rows
print(f"Bounding box data saved to {csv_file_name}")
```

Source Code 2 reads the recorded coordinates of the bounding box from the CSV file, processes each bounding box to download corresponding GeoTIFF images using a retry mechanism, and saves the last successful index to a file to resume the process in case of failure. samgeo, a Python package for segmenting geospatial data with the Segment Anything Model (SAM), primarily handles geospatial operations such as downloading and converting map tiles into GeoTIFF format (https://samgeo.gishub.org/). It provides functionality for working with satellite and map imagery. As specified by the parameter source="ROADMAP", GeoTIFF images could be downloaded in multiple formats depending on the source specified. The ROADMAP option allows for precise linear measurements (e.g., length, width) or more complex analyses of

geometric properties of map features. Roadmaps are less cluttered than aerial images, making it easier to identify and analyze specific features such as building footprints or road intersections. Setting the source parameter to SATELLITE would instruct the function to fetch and convert tiles from an aerial imagery layer, providing detailed, high-resolution views of the Earth's surface, capturing natural features (e.g., vegetation, water bodies) and built environments (e.g., buildings, roads). MHC mapping can be based on either ROADMAP images or SATELLITE images. We first use ROADMAP images to filter statewide images to focus on the built environment, excluding natural features. Next, we employ the SATELLITE version of the filtered images as input for the deep learning detection model. Next, we use the ROADMAP version of the detected MHCs to map the building footprints.

Source Code 2 Script to download GeoTIFF images for specified bounding boxes.

In [ ]:
```
pip install segment-geospatial # Library to install the required dependencies.

import csv # Used for reading CSV files.
import os # Used for creating directories and file path operations.
import time # Used for adding delays between retry attempts.
from samgeo import tms_to_geotiff # Used to download and convert map tiles.

# Create a new folder to save downloaded GeoTIFF images
new_folder_path = "E:/wi/0-1000000"

try:
    os.makedirs(new_folder_path, exist_ok=True)
    print(f"Folder '{new_folder_path}' created successfully.")
except Exception as e:
    print(f"An error occurred while creating the folder: {e}")

# Create a list to store the points' latitude, longitude, and bounding box sizes
csv_file_name = "E:/wi/wi_bounding_boxes.csv"

points_list = []

with open(csv_file_name, 'r') as csvfile:
    csvreader = csv.DictReader(csvfile)  # Use csv.DictReader to access columns by their names
    for row in csvreader:
        x = float(row["x"])
        y = float(row["y"])
        north = float(row["north"])
        south = float(row["south"])
        east = float(row["east"])
        west = float(row["west"])
        points_list.append((x, y, north, south, east, west))

# Set the start and end indices for the slice
start_index = 0 # Adjust this to the desired starting index
end_index = 1000000  # Adjust this to the desired ending index

# Set the maximum number of retries
max_retries = 5

# Load the last successful index from a file
try:
    with open('last_successful_index.txt', 'r') as file:
        last_successful_index = int(file.read().strip())
except FileNotFoundError:
    last_successful_index = start_index

# Function to download GeoTIFF image with retry mechanism
def download_geotiff_with_retry(output_path, bbox):
    retries = 0
    while retries < max_retries:
        try:
```

```
            # Geospatial operation - Download and convert map tiles to a GeoTIFF image
            tms_to_geotiff(output=output_path, bbox=bbox, zoom=19, source="ROADMAP", overwrite=True)
            return True  # Download successful
        except Exception as e:
            print(f"Download failed: {e}. Retrying...")
            retries += 1
            time.sleep(5)  # Add a delay before retrying

    print("Max retries exceeded. Download unsuccessful.")
    return False

# Loop through each point and extract GeoTIFF image
for i, (x, y, north, south, east, west) in enumerate(points_list[last_successful_index:end_index],
                                                     start=last_successful_index):

    # Determine the bounding box for the current point
    bbox = [west, south, east, north]

    # Specify the output path for each GeoTIFF image (customize as needed)
    output_path = f"E:/wi/0-1000000/wi{i}_x{x}_y{y}.tif"

    # Attempt to download with retry mechanism
    if download_geotiff_with_retry(output_path, bbox):
        # Update the last successful index
        last_successful_index = i

        # Save the last successful index to a file
        with open('last_successful_index.txt', 'w') as file:
            file.write(str(last_successful_index))

    # Check if we have reached the end of the slice, and break the loop if necessary
    if i == end_index - 1:
        break
```

Source Code 3 defines the input and output folder paths where the downloaded ROADMAP images are located and where the processed images will be saved. The script reads each TIFF images in batches to avoid memory issues. It reads the images in grayscale, checks for images with identical cell values and skips them, applies a threshold to create a binary mask and dilates the image, saves the processed image as a PNG file. The numeric values for batch processing, threshold value, kernel size for dilation, and the range of white pixel values to be processed should be adjusted experimentally, depending on the values of the input images.

Source Code 3 Script to to threshold images in batches.

In [ ]:

```
import os
import numpy as np
import cv2

# Define the input and output folder paths
input_folder = r'E:\wi\0-1000000'
output_folder = r'E:\wi\0-1000000-thresholded'

# Ensure the destination folder exists, create it if not
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

batch_size = 100
threshold_value = 128  # Adjust this threshold value as needed
kernel = np.ones((3, 3), np.uint8)  # Adjust the kernel size as needed
white_range = (240, 245)  # Adjust the white range as needed

file_names = [file_name for file_name in os.listdir(input_folder) if file_name.endswith(".tif")]
num_files = len(file_names)
```

```
for batch_start in range(0, num_files, batch_size):
    batch_end = min(batch_start + batch_size, num_files)
    batch_file_names = file_names[batch_start:batch_end]

    for file_name in batch_file_names:
        tif_path = os.path.join(input_folder, file_name)
        img = cv2.imread(tif_path, cv2.IMREAD_GRAYSCALE)

        unique_values = np.unique(img)
        if len(unique_values) == 1:
            print(f"Skipping {file_name} - All cell values identical.")
            continue

        mask = img < threshold_value
        img[mask] = 255
        dilation = cv2.dilate(img, kernel, iterations=3)
        output_filename = os.path.splitext(file_name)[0] + ".png"
        output_path = os.path.join(output_folder, output_filename)
        cv2.imwrite(output_path, dilation)

        processed_img = cv2.imread(output_path, cv2.IMREAD_GRAYSCALE)
        processed_img[~np.logical_and(processed_img >= white_range[0],
                                      processed_img <= white_range[1])] = 0
        processed_img[np.logical_and(processed_img >= white_range[0],
                                     processed_img <= white_range[1])] = 255
        cv2.imwrite(output_path, processed_img)
```

Source Code 4 includes functions to check if at least 5 percent of all pixels are white (values close to 255), and at least 15 percent of all pixels are black (values close to 0). Images that meet the specified thresholds are copied to a new destination folder, named "preprocessed".

Source Code 4 Script to filter thresholded images for relevant content.

In [ ]:

```
import os
import cv2
import numpy as np
import shutil

# Source and destination folders
source_folder = r'E:\wi\0-1000000-thresholded'
destination_folder = r'E:\wi\0-1000000-preprocessed'

# Ensure the destination folder exists, create it if not
if not os.path.exists(destination_folder):
    os.makedirs(destination_folder)

# Batch size for processing images
batch_size = 100

# Function to check if the sum of white pixels is at least 5% of all pixels
def has_sufficient_white_pixels(image_path, threshold=0.05):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    total_pixels = img.size
    white_pixels = np.sum(img >= 240)  # Assuming white pixels have values close to 255
    return (white_pixels / total_pixels) >= threshold

# Function to check if the sum of black pixels is at least 15% of all pixels
def has_sufficient_black_pixels(image_path, threshold=0.15):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    total_pixels = img.size
    black_pixels = np.sum(img <= 15)  # Assuming black pixels have values close to 0
    return (black_pixels / total_pixels) >= threshold

# Iterate through PNG images in the source folder in batches
```

```
file_names = [file_name for file_name in os.listdir(source_folder)
              if file_name.endswith(".png")]
num_files = len(file_names)

for batch_start in range(0, num_files, batch_size):
    batch_end = min(batch_start + batch_size, num_files)
    batch_file_names = file_names[batch_start:batch_end]

    for file_name in batch_file_names:
        tif_path = os.path.join(source_folder, file_name)
        if has_sufficient_white_pixels(tif_path, threshold=0.05)
        and has_sufficient_black_pixels(tif_path, threshold=0.20):
            # Copy the image to the destination folder if it meets the threshold
            shutil.copy(tif_path, os.path.join(destination_folder, file_name))
            print(f"Copied {file_name} to the destination folder.")

print("Processing complete.")
```

Source Code 5 sets up and trains a deep learning model to classify images of manufactured housing communities (MHC) versus non-MHC images using TensorFlow and Keras in a Google Colab environment. It starts by installing necessary packages and importing relevant libraries. It then mounts Google Drive to access image directories and loads images of both categories, resizing them to 224x224 pixels. These images are combined into arrays, labeled, and split into 80 percent training and 20 percent testing sets. The pixel values are normalized, and a pre-trained MobileNet model (without its top layers) is loaded and modified by adding custom classification layers (a Flatten layer, a Dense layer with 256 units, a Dropout layer, and a final Dense layer with a sigmoid activation function). The model is compiled with the Adam optimizer and binary cross-entropy loss, trained on the image data for 20 epochs, and finally saved to Google Drive for future use.

Source Code 5 Script to train, test, and save a MobileNet model to classify aerial MHC images.

In [ ]:
```
!pip install tensorflow keras opencv-python

import os
import cv2
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from keras.models import Model
from keras.layers import Flatten, Dense, Dropout
from keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')

# Set the paths to your image directories
mhc_dir = '/content/drive/MyDrive/mhc'
non_mhc_dir = '/content/drive/MyDrive/non_mhc'

# Load the images and labels
mhc_images = []
non_mhc_images = []
labels = []

# Load MHC images
for filename in os.listdir(mhc_dir):
    if filename.endswith(".png"):
        img = cv2.imread(os.path.join(mhc_dir, filename))
        img = cv2.resize(img, (224, 224))  # Resize image to 224x224
        mhc_images.append(img)
        labels.append(1)  # Assign label 1 for MHC images
```

```
# Load non-MHC images
for filename in os.listdir(non_mhc_dir):
    if filename.endswith(".png"):
        img = cv2.imread(os.path.join(non_mhc_dir, filename))
        img = cv2.resize(img, (224, 224))  # Resize image to 224x224
        non_mhc_images.append(img)
        labels.append(0)  # Assign label 0 for Non-MHC images

# Convert lists to numpy arrays
mhc_images = np.array(mhc_images)
non_mhc_images = np.array(non_mhc_images)
labels = np.array(labels)

print("mhc_images shape:", mhc_images.shape)
print("non_mhc_images shape:", non_mhc_images.shape)
print("labels length:", len(labels))

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    np.concatenate((mhc_images, non_mhc_images), axis=0),
    labels,
    test_size=0.4,
    random_state=42
)

# Normalize pixel values to the range [0, 1]
X_train = X_train.astype('float32') / 255
X_test = X_test.astype('float32') / 255

# Load pre-trained MobileNet model (excluding the top layer)
base_model = MobileNet(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze the layers in the base model
for layer in base_model.layers:
    layer.trainable = False

# Add custom classification layers
x = base_model.output
x = Flatten()(x)
x = Dense(256, activation='relu')(x)  # Adjust the number of neurons as needed
x = Dropout(0.5)(x)  # Adding dropout for regularization
x = Dense(1, activation='sigmoid')(x)

# Create the model
model = Model(inputs=base_model.input, outputs=x)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])

# Train the model and store the history
model_history = model.fit(X_train, y_train, epochs=20, batch_size=64, validation_data=(X_test, y_test))

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test loss: {loss}")
print(f"Test accuracy: {accuracy}")

# Save the model for future use
model.save('/content/drive/MyDrive/mobilenet_mhc.h5')
```

Source Code 6 initializes lists to store file names, interpretations, and prediction probabilities. For each image in the folder, it reads and preprocesses the image, makes a prediction using the model, and interprets the result as "YES" if the prediction is 0.5 or higher, and "NO" otherwise. Images predicted as "YES" are copied to an output folder. The results are

then compiled into a DataFrame and saved as a CSV file in a specified location. The code also ensures the output folder exists before processing and provides feedback during processing.

Source Code 6 Script to make predictions using the pre-trained model.

In [ ]:
```
!pip install tensorflow keras opencv-python

import os
import cv2
import numpy as np
import pandas as pd
from keras.models import Model
from keras.models import load_model
import shutil  # Import the shutil library

# Specify the path to the saved model
model_path = 'E:/mobilenet_mhc.h5'

# Load the model
model = load_model(model_path)

# Initialize empty lists to store results
file_names = []
interpretations = []
probabilities = []

# Folder containing the aerial images
folder_path = 'E:/wi/sat'

# Output folder for copied images
output_folder = 'E:/wi/sat-detected'

# Output folder for DataFrame as a CSV file
csv_path = 'E:/wi/mhc_detected.csv'

# Create the output folder if it doesn't exist
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

# Loop through each image in the folder
for file_name in os.listdir(folder_path):
    if file_name.endswith('.tif'):
        image_path = os.path.join(folder_path, file_name)
        print("Processing image:", image_path)

        # Load and preprocess the image
        img = cv2.imread(image_path)
        img = cv2.resize(img, (224, 224))
        img = img.astype('float32') / 255
        img = np.expand_dims(img, axis=0)

        # Make predictions
        predictions = model.predict(img)
        print("Predictions:", predictions)

        # Interpret the predictions
        if predictions[0] >= 0.5:
            interpretation = "YES"
        else:
            interpretation = "NO"

        print("Interpretation:", interpretation)

        # Store results
```

```
        file_names.append(file_name)
        interpretations.append(interpretation)
        probabilities.append(predictions[0])

        # Copy the image to the output folder if interpretation is "YES"
        if interpretation == "YES":
            shutil.copy(image_path, os.path.join(output_folder, file_name))

# Create a DataFrame from the results
result_df = pd.DataFrame({
    'File Name': file_names,
    'Interpretation': interpretations,
    'Probability': probabilities
})

# Save the DataFrame as a CSV file
result_df.to_csv(csv_path, index=False)

print("Results saved to", csv_path)
```

Source Code 7 takes a representative image from each MHC and expands the original boundaries by three times the width and height to cover the entire community. A function is defined to determine if two bounding boxes overlap. If an overlap is found, the script assigns the same group id (community id) to the overlapping bounding boxes, and the resulting DataFrame with the grouped bounding boxes is saved to a new CSV file. Next, we retrieve the aerial, ROADMAP version of the image, following the method shown in Source Code 1, as it facilitates building footprint extraction.

Source Code 7 Script to expand the original boundaries and group overlapping observations.

In [ ]:

```
import csv
import math
import os
import pandas as pd

# Load the CSV file into a DataFrame
input_csv_file = 'mhc-inspected-duplicate-cleaned.csv'

# Specify the file path where you want to save the CSV file
output_csv_file = os.path.join(folder_path, 'mhc-inspected-duplicate-cleaned-grouped.csv')

df = pd.read_csv(input_csv_file, sep=',')

# Calculate the new bounding box boundaries
df['width_deg'] = df['east'] - df['west']
df['height_deg'] = df['north'] - df['south']

df['north_w'] = df['north'] + (3 * df['height_deg'])
df['south_w'] = df['south'] - (3 * df['height_deg'])
df['east_w'] = df['east'] + (3 * df['width_deg'])
df['west_w'] = df['west'] - (3 * df['width_deg'])

# Function to check overlap between two bounding boxes
def check_overlap(bbox1, bbox2):
    return not (bbox1['west_w'] > bbox2['east_w'] or
                bbox1['east_w'] < bbox2['west_w'] or
                bbox1['north_w'] < bbox2['south_w'] or
                bbox1['south_w'] > bbox2['north_w'])

# Create a new column to group overlapping observations
df['group_id'] = None
group_counter = 0

# Iterate through each observation
for i in range(len(df)):
```

```
    # If the observation is not yet grouped
    if df.at[i, 'group_id'] is None:
        # Create a group for this observation
        df.at[i, 'group_id'] = group_counter
        group_counter += 1

    # Check for overlap with other observations
    for j in range(i+1, len(df)):
        if check_overlap(df.loc[i], df.loc[j]):
            # If there is an overlap, assign the same group ID
            if df.at[j, 'group_id'] is None:
                df.at[j, 'group_id'] = df.at[i, 'group_id']

# Save the DataFrame to a CSV file
df.to_csv(output_csv_file, index=False)

print("DataFrame saved to", output_csv_file)
```

Source Code 8 takes the downloaded GeoTIFF images that belong to the same group (i.e., same community as defined by overlapping expanded boundaries) and plots them on a common "canvas". This Python script processes GeoTIFF images in a specified folder, groups them based on a 'group' identifier in their filenames, and then overlays the images within each group into a single composite image. The script iterates over the TIFF files to extract and group them by their 'group' identifier, and then for each group, it calculates the overall geographic extent and overlays the images using their spatial information. The resulting composite image for each group is saved as a high-resolution PNG file in the output directory. The script uses rasterio and matplotlib for image processing and plotting. rasterio depends on GDAL for its core functionality. The script includes a delay of 5 seconds between processing each group to manage resource usage and prints the status of saved images. The script will consume significant RAM over time, likely necessitating a system restart after about two hours. Saving the GeoTIFF files in PNG format results in the images losing their geographical identifiers. Once we have grouped images saved as high-resolution PNG files, we threshold them, adapting the Source Code 3. We then reassign their geographical coordinates to convert them back to GeoTIFF files in Google Colab environment.

Source Code 8 Script to plot GeoTIFF images that belong to the same group on a common canvas.

In [ ]:
```
conda install gdal -c conda-forge

import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import rasterio
from rasterio.plot import plotting_extent
import time

folder_path = r'E:\wi\mhp-inspected-duplicate-cleaned-grouped'
output_folder_path = r'E:\wi\mhp-inspected-duplicate-cleaned-grouped-overlayed'

# Check if the directory already exists; if not, create it
if not os.path.exists(output_folder_path):
    os.makedirs(output_folder_path)

tiff_files = [f for f in os.listdir(folder_path) if f.endswith('.tif')]

file_names_with_groups = {}  # Dictionary to store group names and associated file names

for tiff_file in tiff_files:
    file_name = os.path.splitext(tiff_file)[0]  # Remove the file extension
    parts = file_name.split('_')  # Split the file name by underscores
    group = None

    # Iterate through parts to find the group name
    for part in parts:
```

```
        if part.startswith('group'):
            group = part.split('group')[1]
            break

    if group:
        file_names_with_groups.setdefault(group, []).append(tiff_file)

# Print the list of file names and their associated group names
for group, file_names in file_names_with_groups.items():
    print(f"Group {group}: {file_names}")

# Load the last saved group or start from the beginning
last_processed_group = 0

# Iterate over each group and overlay images
for group, tiff_files in list(file_names_with_groups.items())[last_processed_group:]:
    # Initialize variables to store overall extent for this group
    overall_min_x = float('inf')
    overall_min_y = float('inf')
    overall_max_x = float('-inf')
    overall_max_y = float('-inf')

    # Initialize plot for this group
    fig, ax = plt.subplots(figsize=(10, 10))

    # Iterate over each GeoTIFF image in this group and plot them
    for tiff_file in tiff_files:
        tiff_path = os.path.join(folder_path, tiff_file)

        # Open the GeoTIFF file and get the spatial information
        with rasterio.open(tiff_path) as src:
            # Get the geographic extent of the image
            extent = plotting_extent(src)

            # Update overall extent for this group
            overall_min_x = min(overall_min_x, extent[0])
            overall_min_y = min(overall_min_y, extent[2])
            overall_max_x = max(overall_max_x, extent[1])
            overall_max_y = max(overall_max_y, extent[3])

            # Read the image and geographic information
            img = src.read(1)  # Assuming a single band image

            # Get the affine transformation to correctly position the image
            transform = src.transform

            # Plot the image with correct geographic positioning
            ax.imshow(img, extent=[transform[2], transform[2] + transform[0] * src.width,
                                   transform[5] + transform[4] * src.height, transform[5]],
                      cmap='gray', alpha=1)

    # Set the overall plot extent for this group
    ax.set_xlim(overall_min_x, overall_max_x)
    ax.set_ylim(overall_min_y, overall_max_y)

    # Hide the axes and labels
    ax.axis('off')

    # Set the DPI for the saved image to achieve the desired resolution
    desired_dpi = 900  # Adjust this value to achieve at least 5000 by 5000 pixels
    output_file_path = os.path.join(output_folder_path, f'overlayed_image_group{group}.png')
    plt.savefig(output_file_path, dpi=desired_dpi, bbox_inches='tight', pad_inches=0, transparent=True)

    print(f'Image saved for Group {group} as:', output_file_path)
```

```
    # Delay for 5 seconds
    time.sleep(5)
```

Source Code 9 converts PNG images into GeoTIFF files, assigning appropriate geographical coordinates. The process starts by installing the rasterio library and mounting Google Drive to access files. The script then reads a CSV file containing bounding box coordinates and group IDs, preserving specific columns. It summarizes these variables by grouping the data based on group id to find the maximum and minimum coordinates for each group. The script extracts the group id from each PNG image file name in a specified directory, calculates the spatial resolution and affine transformation matrix for the images, and reads the image data. Using rasterio, it creates new GeoTIFF files with the geographical coordinates based on the summarized data, saving them in a specified output directory. The process iterates over each image file, ensuring each resulting GeoTIFF file retains the spatial information of the original image.

Source Code 9 Script to convert PNG images into GeoTIFF images with coordinates in Google Colab environment.

In [ ]:
```
!pip install rasterio

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import numpy as np
import os
import pandas as pd
import rasterio
import time

# File path
file_path = r'/content/drive/MyDrive/ne/mhp-inspected-duplicate-cleaned-grouped.csv'

# Columns to preserve
columns_to_preserve = ['north_w', 'south_w', 'east_w', 'west_w', 'group_id']

# Read the CSV file and select the desired columns
df = pd.read_csv(file_path, usecols=columns_to_preserve)

# Group by 'group_id' and summarize the variables
summary_data = df.groupby('group_id').agg(
    max_north_w=pd.NamedAgg(column='north_w', aggfunc='max'),
    min_south_w=pd.NamedAgg(column='south_w', aggfunc='min'),
    max_east_w=pd.NamedAgg(column='east_w', aggfunc='max'),
    min_west_w=pd.NamedAgg(column='west_w', aggfunc='min')
).reset_index()

print(summary_data)

import re

def extract_group_id(file_name):
    match = re.search(r'group(\d+)', file_name)
    if match:
        group_id = int(match.group(1))
        return group_id
    else:
        return None

# Directory containing images
image_directory = '/content/drive/MyDrive/ne/mhp-inspected-duplicate-cleaned-grouped-overlayed-thresholded/'

# Get a list of all image files in the directory
image_files = [f for f in os.listdir(image_directory) if f.endswith('.png')]

# Iterate over each image file
```

```
for image_file in image_files:
    # Image file path
    image_path = os.path.join(image_directory, image_file)

    # Extract group_id from the image file name
    file_name = image_file.split('/')[-1]
    group_id = int(float(extract_group_id(file_name)))

    # Create a new GeoTIFF file
    output_directory = image_directory + '-geotiff'
    output_path = os.path.join(output_directory, f'group{group_id}.tif')

    # Find lon_max, lon_min, lat_max, and lat_min using the group_id
    lon_max = summary_data[summary_data['group_id'] == group_id]['max_east_w'].values[0]
    lon_min = summary_data[summary_data['group_id'] == group_id]['min_west_w'].values[0]
    lat_max = summary_data[summary_data['group_id'] == group_id]['max_north_w'].values[0]
    lat_min = summary_data[summary_data['group_id'] == group_id]['min_south_w'].values[0]

    # Read the image
    image = rasterio.open(image_path)

    # Get image width and height in pixels
    image_width = image.width
    image_height = image.height

    # Calculate the spatial resolution
    resolution_x = (lon_max - lon_min) / image_width
    resolution_y = (lat_max - lat_min) / image_height

    # Calculate the affine transformation matrix
    transform = rasterio.Affine(resolution_x, 0, lon_min,
                                0, -resolution_y, lat_max)

    # Read the RGB image data
    image_data = image.read()  # Read all bands

    # Create an empty array for the new GeoTIFF
    new_image = np.zeros((image_height, image_width), dtype=image_data.dtype)

    # Assign the image data to the new GeoTIFF array
    new_image[:, :] = image_data[:, :]

    # ... (previous code)
    # Get CRS in rasterio format
    crs = rasterio.crs.CRS.from_epsg(4326)

    # Write the new GeoTIFF file from RGB input
    with rasterio.open(output_path, 'w', driver='GTiff', width=image_width,
                       height=image_height, count=image.count, dtype=image_data.dtype,
                       crs=crs, transform=transform) as dst:
        for band in range(image.count):
            dst.write(image_data[band], band + 1)

    time.sleep(5)
```

Source Code 10 converts the prepared GeoTIFF files into shapefiles. This code is designed to be run inside ArcGIS Pro environment, and before running, a base map should be added in ArcGIS Pro. The script sets up the ArcPy workspace to a specified directory containing GeoTIFF files and defines an output directory for the resulting shapefiles, creating it if it does not exist. The script iterates through each GeoTIFF file in the workspace, extracting the base name of each file to use as a group name. For each GeoTIFF, it creates a corresponding directory and shapefile path, imports the GeoTIFF as a raster object, and converts the raster to a polygon feature class, saving it as a shapefile. Additionally, it adds a new text field named "Group" to the shapefile and populates it with the group name. Finally, the script maintains a list of the paths to the created shapefiles for potential further processing.

Source Code 10 Script to convert GeoTIFF files into shapefiles in ArcGIS Pro environment.

In [ ]:
```
import arcpy
import os

# Set the workspace: Replace with the actual path to your workspace
workspace = r"E:/wi/mhp-inspected-duplicate-cleaned-grouped-overlayed-thresholded-geotiff"
arcpy.env.workspace = workspace

# Output directory for shapefiles
output_directory = r"E:/wi/shapefile"  # Replace with the desired path to save the output shapefiles

# Create the output directory if it doesn't exist
if not os.path.exists(output_directory):
    os.makedirs(output_directory)

# Create an empty list to store the paths of intermediate shapefiles
intermediate_shapefiles = []

# Iterate through GeoTIFF files in the workspace
for geotiff_file in arcpy.ListFiles("*.tif"):
    # Create the output shapefile path for each group
    group_name = os.path.splitext(os.path.basename(geotiff_file))[0]  # Use file name as group name
    group_output_dir = os.path.join(output_directory, f"group_{group_name}")
    group_shapefile_path = os.path.join(group_output_dir, f"group_{group_name}.shp")

    # Create the group's output directory if it doesn't exist
    if not os.path.exists(group_output_dir):
        os.makedirs(group_output_dir)

    # Import the GeoTIFF into a raster object
    raster = arcpy.sa.Raster(os.path.join(workspace, geotiff_file))

    # Convert the raster to a polygon feature class
    arcpy.conversion.RasterToPolygon(raster, group_shapefile_path, "NO_SIMPLIFY", "VALUE")

    # Add a field to store the group information
    arcpy.management.AddField(group_shapefile_path, "Group", "TEXT")
    arcpy.management.CalculateField(group_shapefile_path, "Group", f"'{group_name}'", "PYTHON3")

    # Append the intermediate shapefile path to the list
    intermediate_shapefiles.append(group_shapefile_path)
```

After a visual inspection of individual shapefiles created and removing or editing any overlapping ones, we write another Python script, utilizing the ArcPy library, to processes and merge shapefiles in ArcGIS Pro. The script first accesses the current ArcGIS Pro project and retrieves the active map. Then, it iterates through each layer in the map, constructing file paths for each layer's shapefile and storing these paths in a list. The script defines an output location for the merged shapefile and then merges the shapefiles from the list into a single shapefile saved at the specified output location. This merged shapefile can then be used in a new GIS project for further analysis or processing. To clean the file utilizing the ArcPy library, we write a script to process the shapefile by calculating the geometries of its features and removing those

with areas under 100 square feet. By performing the aforementioned steps, we prepare raw shapefiles of MHCs, which include both the detected MHC structures and the surrounding non-MHC structures. We then clean the surrounding non-MHC structures using the base map and parcel map underlays.